

Complejidad Temporal

Pablo Castro
UNRC - Computabilidad y Complejidad

Tiempo de Ejecución

En computabilidad nos preocupamos por el siguiente problema:

Dado in problema P , es computable?

En complejidad ya trabajamos con problemas computables, la pregunta es:

Hay algoritmos eficientes para resolver P ?

Una cuestión es cuál modelo de computación usamos para evaluar si un algoritmo es eficiente o no?

Tiempo de Ejecución como un Función

Dado un algoritmo M su tiempo de ejecución viene dado por una función:

$$T_M : \mathbb{N} \rightarrow \mathbb{R}^{\geq 0}$$

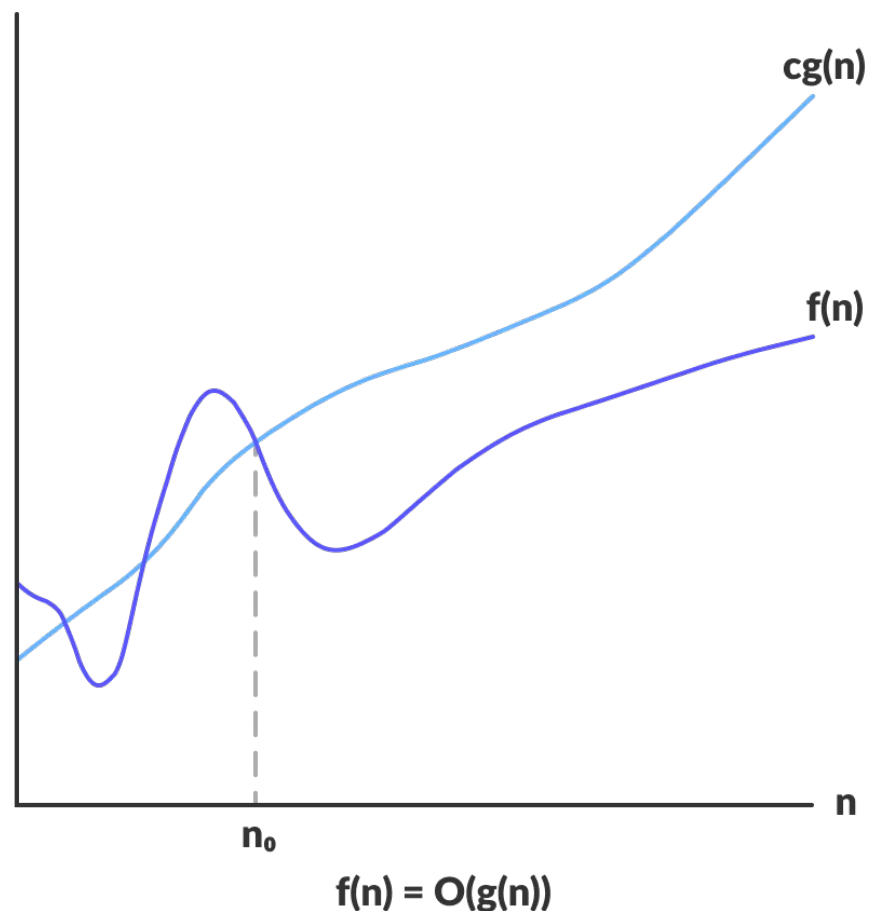
Que dice, para una entrada de tamaño n , cuantos pasos realiza M en el **peor caso**, o **caso promedio**.

- En general asumimos que T es monótona,
- M es una MT que termina en todas sus entradas,
- En general hacemos un análisis en el **peor caso**.
- Nos interesa analizar como T crece cuando las entradas se hacen más grandes.

Notación O

La notación O (big oh) nos permite analizar el tiempo de ejecución de los algoritmos.

$$f(n) \in O(g(n)) \Leftrightarrow \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) \leq c * g(n)$$



Intuitivamente: *para entradas grandes g crece más rápido que f modulo constantes multiplicativas.*

Notación o (little oh)

La o (little oh) es una versión estricta de la O :

$$f(n) \in o(g(n)) \Leftrightarrow \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) < c * g(n)$$

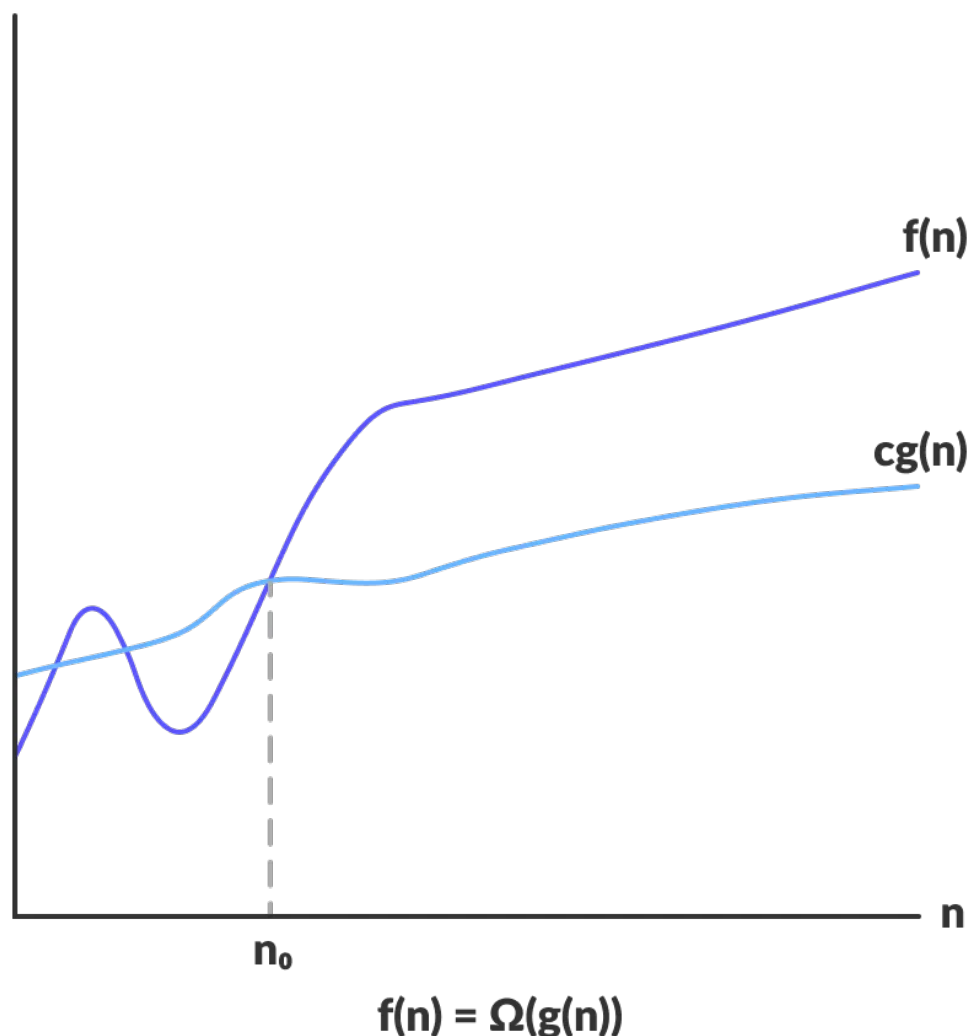
La o dice que $g(n)$ crece estrictamente más rápido que f . Es decir, no tenemos $f(n) \in o(f(n))$.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \text{ si } f(n) \in o(g(n))$$

Notación Ω

La Ω (omega) es sirve para hablar de cotas por abajo.

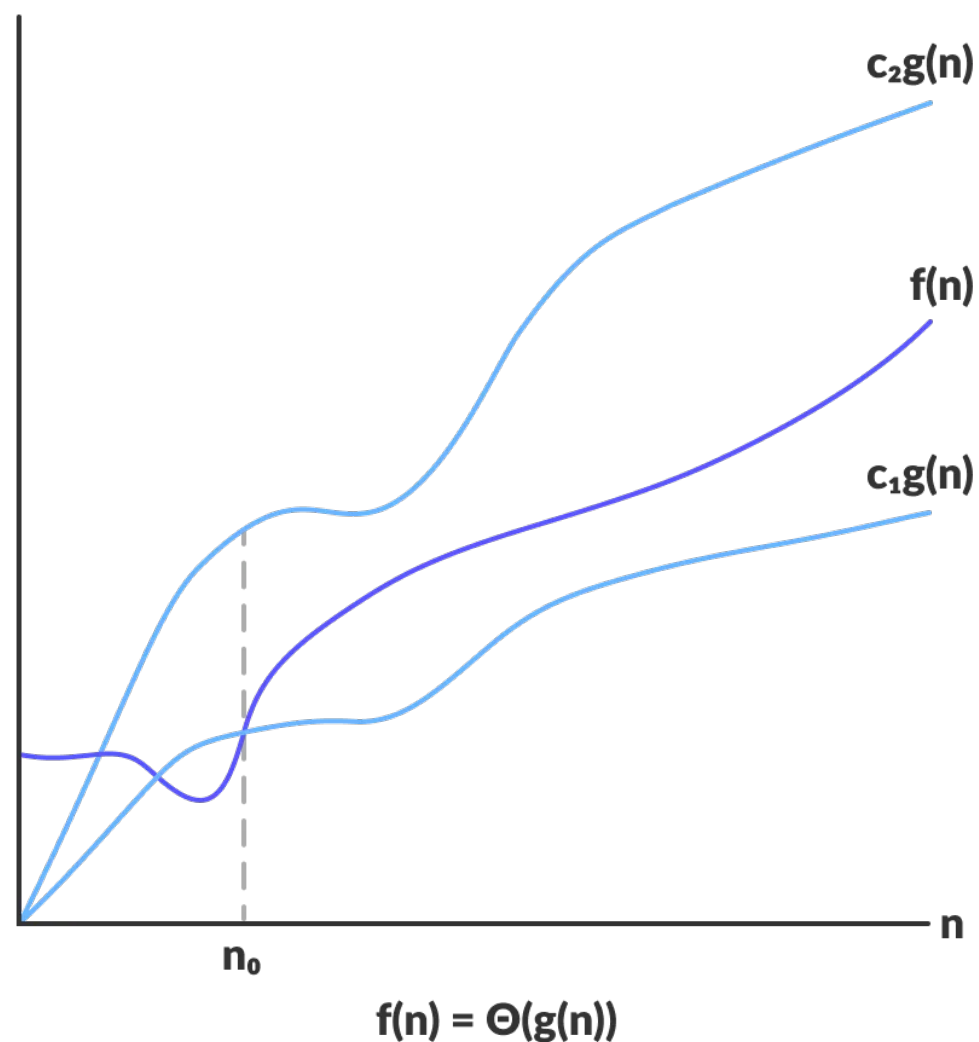
$$f(n) \in \Omega(g(n)) \Leftrightarrow \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) \geq c * g(n)$$



Intuitivamente: *para entradas grandes f crece más rápido que g modulo constantes multiplicativas.*

Notación Θ

La Θ (theta) sirve para ver que dos funciones crecen de la misma forma:



Intuitivamente: *para entradas grandes f y g crecen igual modulo constantes multiplicativas.*

Análisis de Tiempo de Ejecución

Supongamos el siguiente lenguaje:

$$A = \{0^k 1^k \mid k \geq 0\}$$

Podemos hacer la siguiente MT para decidirlo.

Dado el input w :

1. Se recorre la cinta, si se encuentra “10” se rechaza,
2. Se vuelve al principio, se tacha el primer “0” y se busca el primer “1” y se tacha,
3. Se repite el paso 2 hasta que:
 - I. No queden 0s y 1s, en tal caso se acepta,
 - II. Queden solo 0s o solo 1s, en tal caso se rechaza

Análisis de Tiempo de Ejecución

Su tiempo ejecución depende de la entrada:

- ➡ Para la entrada $0111\dots1110$ con una pasada se rechaza, es decir, se realizan $O(n)$ pasos.
- ➡ Para entradas del tipo $000\dots000111\dots111$ ($n/2$ 1's y 0s). Hacemos aproximadamente $2 * n + n/2 * n$ pasos.

Es decir, en el peor caso es $O(n^2)$ y en el mejor caso es $O(n)$.

En lo siguiente nos enfocaremos en el análisis en el peor caso.

Análisis detallado de Tiempo de Ejecución

Para una entrada de longitud n , analicemos cuántos pasos realiza la máquina:

$O(n)$ pasos
($2 * n$)

Dado el input w :

n pasos

1. Se recorre la cinta, si se encuentra "10" se rechaza,

2. Se vuelve al principio, se tacha el primer "0" y se busca el primer "1" y se tacha,

3. Se repite el paso 2 hasta que:

I. No queden 0s y 1s, en tal caso se acepta,

II. Queden solo 0s o solo 1s, en tal caso se rechaza

$n/2$ veces

En total tenemos $O(n) + O(n/2 * n) \in O(n^2)$ pasos, es decir, es $O(n^2)$

Análisis de Tiempo de Ejecución

La solución es $O(n^2)$, consideremos la siguiente solución:

Dada la entrada w se procede de la siguiente forma:

1. Se recorre la cinta, si se encuentra “10” se rechaza,
2. Repetir mientras haya 0's o 1's
 - a. Se cuenta la cantidad de 0's y 1', si es impar se rechaza.
 - b. Cada dos 0's tachar de forma alternante 0's, y lo mismo para los 1's,
3. Si no quedan 0s y 1's se acepta, sino se rechaza

Ejemplo de Corrida

Supongamos que tenemos:

000...111... (25 0s y 25 1s)

Entonces:

- ➡ El paso (1) se verifica que está en la forma correcta
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 12 0's y 12 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 6 0's y 6 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 3 0's y 3 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 2 0's y 2 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 1 0's y 1 1's, termina.

Análisis del Algoritmo

Analicemos este algoritmo:

Dada la entrada w se procede de la siguiente forma:

1. Se recorre la cinta, si se encuentra "10" se rechaza,

2. Repetir mientras haya 0's o 1's

$O(n)$ pasos

I. Se cuenta la cantidad de 0's y 1's, si es impar se rechaza.

$O(\log n)$
pasos

II. Cada dos 0's tachar uno de ellos, y lo mismo para los 1's,

$O(n)$ pasos

3. Si no quedan 0's y 1's se acepta, sino se rechaza

En total realizamos $O(n * \log n)$ pasos. Mejor que la otra solución.

Clases de Complejidad

Permite clasificar los problemas en base a cotas superiores de algoritmos que los resuelven:

$\text{Time}(t(n)) = \{L \mid L \text{ se puede decidir con una MT en } O(t(n)) \text{ pasos}\}$

Por ejemplo:

- $\text{Time}(O(n^2))$, problemas para los cuales existen un algoritmo que en el peor caso realizan $c * n^2$ pasos, para alguna constante c (sorting, por ejemplo)

Qué se estudia en complejidad:

- Dado un problema, pertenece a $\text{Time}(O(t(n)))$?
- Habrá problemas que pertenecen a $\text{Time}(O(t(n)))$ pero no a $\text{Time}(O(t'(n)))$

Máquinas de una cinta vs Máquinas de muchas cintas

Podemos mejorar nuestra solución anterior con una MT de dos cintas:

Dada la entrada w :

Hace 4 $O(n)$
fases, es $O(n)$

1. Se recorre la cinta, si se encuentra "10" se rechaza,
2. Recorremos la cinta una segunda vez, y copiamos los ceros a la segunda cinta,
3. Recorremos la cinta otra vez, y cada vez que encontramos un 1, tachamos un cero en la segunda cinta. Si no tenemos más ceros rechazamos
4. Si todos los ceros de la segunda cinta fueron tachados aceptamos, si quedan ceros rechazamos

La Clase P

Los algoritmos que corren en $O(n^k)$ para una entrada de longitud n se dicen **tratables**.

	n=10	n=20	n=30	n=40	n=50
n	0.00001 segs	0.00002 segs	0.00003 segs	0.00004 segs	0.00005 segs
n^2	0.0002 segs	0.0004 segs	0.0009 segs	0.0016 segs	0.0025 segs
n^3	0.1 segs	3.2 segs	24.3 segs	1.7 min	15.2 min
2^n	0.001 segs	1.0 segs	17.9 min	12.7 dias	35.6 años

Definición de la clase de complejidad P

Podemos definir la clase de los algoritmos cuyo tiempo ejecución es polinomial.

$$P = \bigcup_{k>0} \text{Time}(O(n^k))$$

Es decir, aquellos problemas para los cuales existen algoritmos en $O(n^k)$, para algún k , que los resuelven.

- ✓ La clase P es invariante con respecto a los modelos de computación deterministas.
- ✓ Se consideran eficientes, si tenemos una solución polinomial, en general, el avance en el hardware va a tener un impacto.

Modelos de Computación

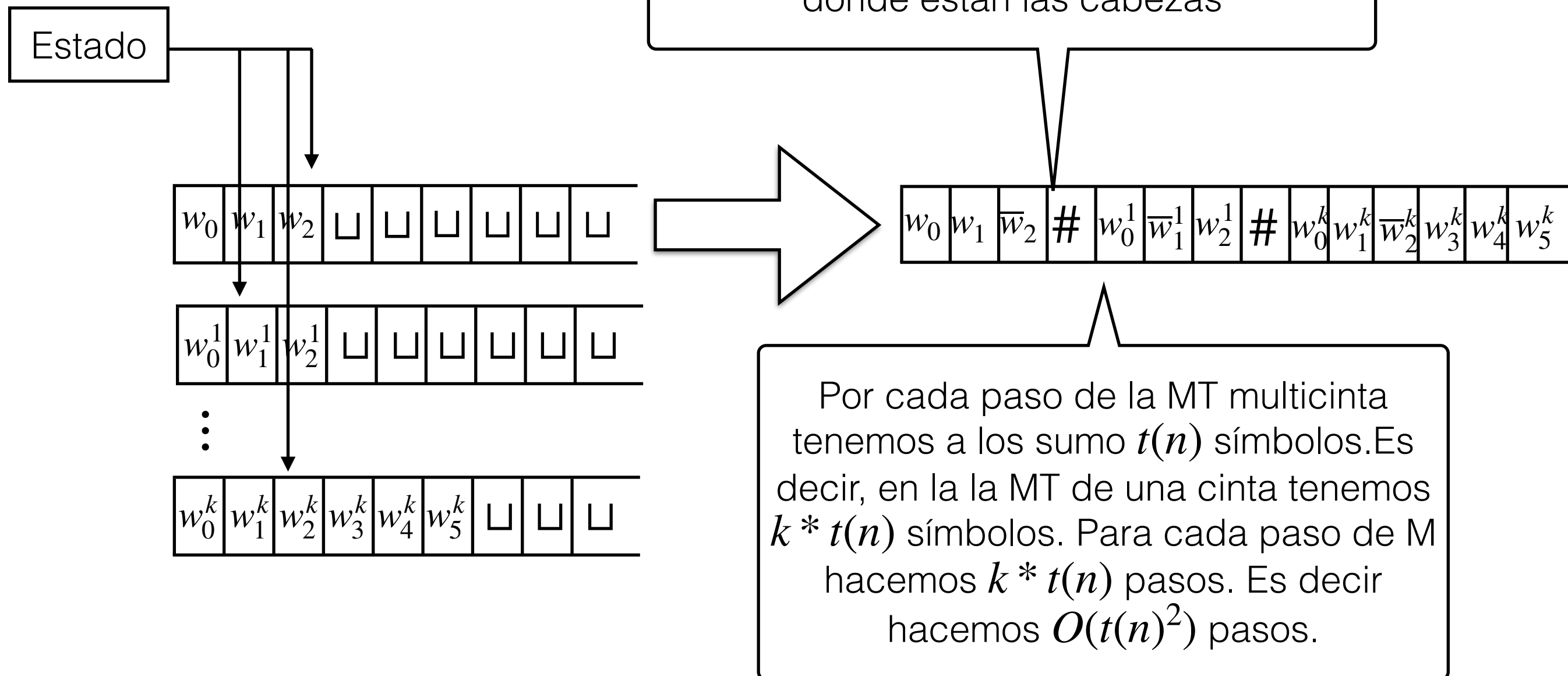
Como hemos visto existen varios modelos de computación:

- Máquinas de Turing deterministas con una sola cinta,
 - Máquinas de Turing deterministas con varias cintas,
 - Máquinas de Turing deterministas con cinta infinita en ambos lados
 - Máquinas de Turing no-deterministas.
- 
- Se pueden simular en tiempo polinomial

La clase Time solo toma como referencia las máquinas deterministas

MT con muchas cintas vs MT con una cinta

Cualquier MT con muchas cintas que hace $O(t(n))$ pasos puede ser traducida a una MT con una cinta que la simula en $O(t(n)^2)$ pasos:



Tesis de Church-Turing Extendida

Recordemos la tesis de Church-Turing.

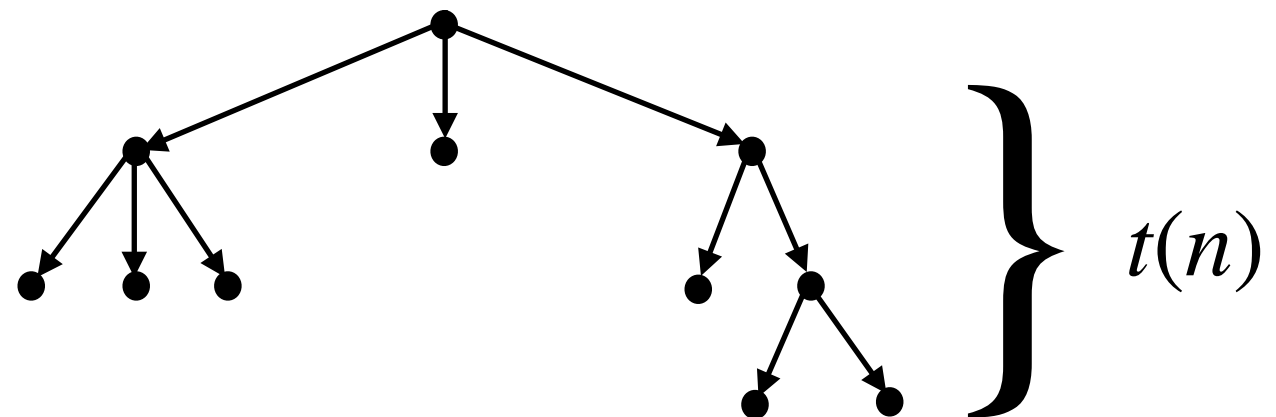
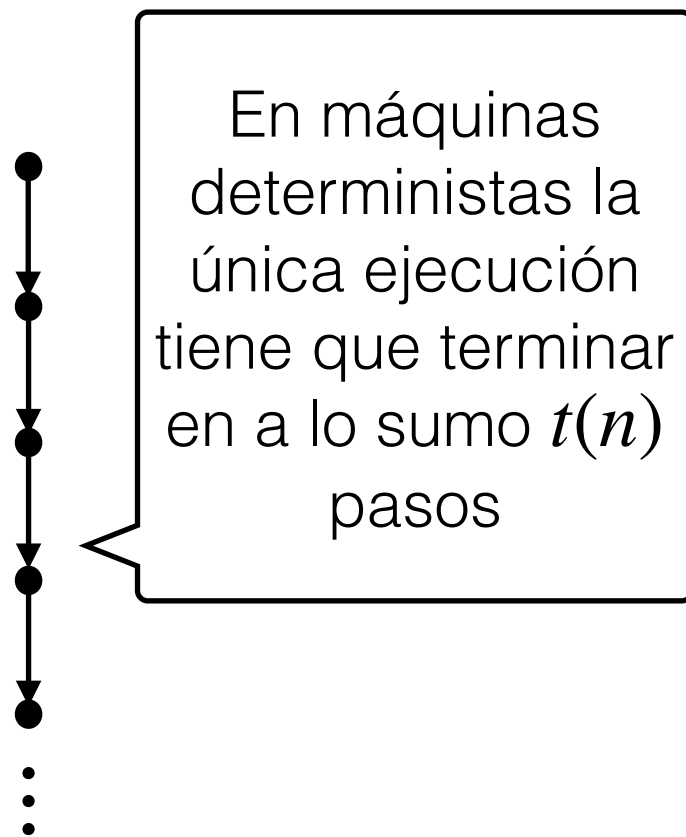
Si un problema es computable por algún modelo de computación, entonces es computable en todos los otros.

Generalmente se considera una versión extendida de la tesis de Church-Turing:

*Si un problema es decidible en tiempo $O(t(n))$ por algún modelo razonable de computación (**determinista**), entonces es decidible en $O(t(n)^k)$ en cualquier otro modelo razonable de computación (**determinista**).*

Tiempo de Ejecución en MT no-deterministas

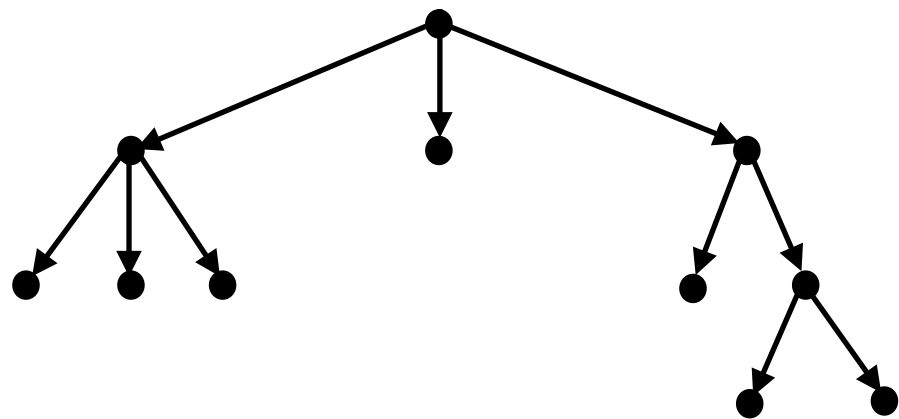
Definición de tiempo de ejecución en máquinas de Turing
no-deterministas



En máquinas deterministas todas las ejecuciones tienen que realizar a lo sumo $t(n)$ pasos

Simulando Máquinas No-Deterministas

Cualquier MT no-determinista que hace $O(t(n))$ pasos puede ser simulada por una MT determinista que hace $2^{O(t(n))}$ pasos.



Recordemos que la simulamos con un BFS, por niveles

- Como la MT termina, el árbol tiene a lo sumo $|b|^{t(n)}$ nodos
- En el peor caso los tenemos que recorrer todos,
- $b^{t(n)} \leq (2^c)^{t(n)} = 2^{c \cdot t(n)} \in 2^{O(t(n))}$

Clase P : Un Ejemplo

Veamos un ejemplo de problema en P :

Path = $\{\langle G, v, w \rangle \mid G \text{ es un grafo con un camino de } v \text{ a } w\}$

Algoritmo ineficiente:

Es $O(n!)$, siendo n la cantidad de nodos del grafo

Dado un grafo codificado con matriz de adyacencia:

Para $i=1,2,3,\dots,\#V$

- ➡ Se generan todos los posibles caminos de longitud i , con empiezan con v y terminan con w
- ➡ Si alguno de estos está en el grafo, se acepta

Si no se encontró, se rechaza.

Ejemplo (cont.)

Veamos una solución mejor:

Dado un grafo representado con matrices de adyacencia:

- ➡ Se marca v ,
- ➡ Mientras haya nodos sin marcar
 - ➡ Recorrer los arcos de G , para cada arco (a,b) con a marcado y b no, se marca b .
 - ➡ Si se marca w , se acepta.
- ➡ Se rechaza.

Se recorren todos los nodos, por cada uno se hace a lo sumo la cantidad de arcos, es decir, el algoritmo es:

$O(|V|*|E|)$ pasos, es decir, es **polinomial**.

Otro ejemplo

Veamos otro ejemplo en P :

Coprimos = $\{\langle x, y \rangle \mid x \text{ es coprimo de } y\}$

Podríamos intentar el siguiente algoritmo:

- ➡ Dados dos números x e y en binario:
 - ➡ Para todo $1 < z \leq \min(x, y)$ hacer
 - ➡ Si $z \mid x$ y $z \mid y$, terminar y rechazar
 - ➡ Sino aceptar

Los números vienen representados en binario. Es decir, si tenemos números de n bits, hacemos 2^n pasos

Otro Ejemplo (cont.)

Veamos otra posible implementación:

- ➡ Dados dos números x e y en binario:
 - ➡ Repetir hasta que $y = 0$
 - ➡ $x = x \bmod y$
 - ➡ intercambiar x e y
 - ➡ Si $x = 1$ aceptar, sino rechazar

Veamos el tiempo de ejecución.

- En cada paso x es dividido al menos por 2,
- Es decir, si los números tienen n bits, toma n pasos,
- La división se puede calcular en tiempo polinomial $p(n)$
- Es decir, es $n * p(n)$ que es polinomial.

Gramáticas Libre de Contexto

Veamos el siguiente ejemplo:

$$GLC = \{ \langle G, w \rangle \mid w \text{ es generado por la gramática libre de contexto } G \}$$

La idea es usar programación dinámica y la forma normal de Chomsky, es decir, producciones del estilo:

$$A \rightarrow BC \qquad A \rightarrow a \qquad A \rightarrow \epsilon$$

La idea es, dado $w = w_0 w_1 \dots w_{n-1}$, construir una matriz:

$$M[i][j] \text{ no terminal que genera desde } i \text{ a } j$$

Algoritmo

El algoritmo es el siguiente:

$D =$ “On input $w = w_1 \cdots w_n$:

1. For $w = \varepsilon$, if $S \rightarrow \varepsilon$ is a rule, *accept*; else, *reject*. $\llbracket w = \varepsilon \text{ case} \rrbracket$
2. For $i = 1$ to n : $\llbracket \text{examine each substring of length 1} \rrbracket$ n pasos
3. For each variable A : v variables
4. Test whether $A \rightarrow b$ is a rule, where $b = w_i$.
5. If so, place A in $table(i, i)$. n pasos
6. For $l = 2$ to n : $\llbracket l \text{ is the length of the substring} \rrbracket$ n pasos
7. For $i = 1$ to $n - l + 1$: $\llbracket i \text{ is the start position of the substring} \rrbracket$ n pasos
8. Let $j = i + l - 1$. $\llbracket j \text{ is the end position of the substring} \rrbracket$
9. For $k = i$ to $j - 1$: $\llbracket k \text{ is the split position} \rrbracket$
10. For each rule $A \rightarrow BC$:
11. If $table(i, k)$ contains B and $table(k + 1, j)$ contains C , put A in $table(i, j)$.
12. If S is in $table(1, n)$, *accept*; else, *reject*.”

Este algoritmo es $O(|v| + n^2)$

Ejercicios

1. Decir cuales son verdaderas y cuales falsas:

a. $2n \in O(n)$

b. $n^2 \in O(n)$

c. $n^2 \in O(n \log^2 n)$

d. $3^n \in 2^{O(n)}$

e. $2^n \in o(3^n)$

f. $1 \in o(1/n)$

Ejercicios

2. Implementar los algoritmos para ver si dos números son coprimos. Probarlos con números de 8 bits, 16 bits, 32 bits y 64 bits.
3. Demostrar que P es cerrada con respecto a la unión, intersección, y a la concatenación.
4. Será P cerrada con respecto al complemento, fundamente.

La Clase NP

Al igual que hicimos con la clase P podemos utilizar MT no deterministas para categorizar los problemas según su complejidad.

$NTIME(t(n)) = \{L \mid L \text{ es un lenguaje decidido por una MT no determista en } t(n) \text{ pasos} \}$

De la misma forma que hicimos de la forma anterior podemos definir:

$$NP = \bigcup_{k>0} NTIME(n^k)$$

Todos los problemas resolubles por una máquina de Turing no-deterministas en tiempo polinomial

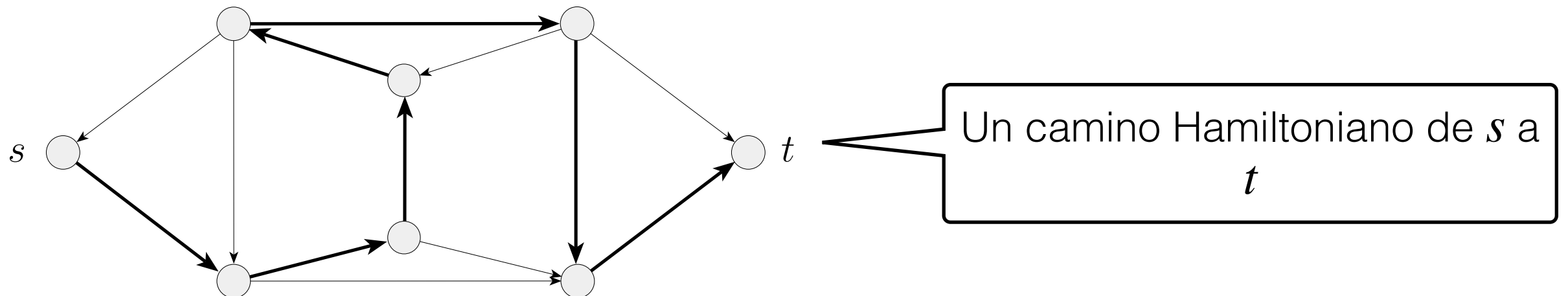
Caminos Hamiltonianos

Dado un grafo dirigido $G = (V, E)$:

Un camino Hamiltoniano es un camino que pasa exactamente una vez por cada nodo.

Definimos:

$\text{HAMPATH} = \{ \langle G, s, v \rangle \mid G \text{ tiene un camino Hamiltoniano de } s \text{ a } t \}$



Una Solución No Determinista

Veamos cómo sería una MT no determinista que resuelva este problema:

Dado un grafo $G = \langle V, E \rangle$ en la cinta de entrada:

1. Escribimos de forma no-determinista $|V|$ nodos
2. Verificamos que estos nodos estén conectados, y son todos distintos, si es así aceptamos, sino rechazamos

Veamos que el algoritmo tiene dos etapas:

- ➡ En la primera etapa (1) se “adivina” no determinísticamente una posible solución,
- ➡ En la segunda etapa se verifica esta solución.

Verificabilidad Polinomial

Decimos que una cadena es un **certificado** de un problema, si con esa cadena podemos ver la solución al problema.

El problema HAMPATH tiene dos características importantes:

1. Posee certificados cortos: *un camino Hamiltoniano se puede describir en menos espacio que el grafo.*
2. Los certificados se pueden verificar rápidamente: Dado un camino podemos verificar en tiempo lineal si es una solución o no.

Los problemas que están en NP tienen esta característica poseen certificados polinomiales que se pueden verificar polinomialmente

Verificadores y Certificados

Dado un lenguaje A , un verificador para A es una MT V tal que:

$$A = \{w \mid \exists c : V \text{ acepta } (w, c)\}$$

Intuitivamente, M verifica que c certifica una solución al problema w .

Un verificador se dice polinomial si V corre en $O(n^k)$ y los certificados son polinomiales con respecto a la entrada:

$$A = \{w \mid \exists c : |c| \leq |w|^k \text{ y } V \text{ acepta } (w, c)\}$$

Para algún $k > 0$ y MT V que corre en $O(n^k)$.

Ejemplos

Veamos un ejemplo:

$$\text{HAMPATH} = \{ \langle G, s, t \rangle \mid \exists p : \forall \text{ verifica que } p \text{ es un camino de } s \text{ a } t \}$$

Los caminos Hamiltonianos tienen la misma cantidad de nodos que el grafo, y pueden ser verificados rápidamente.

$$\text{COMP} = \{ x \mid \exists p, q > 1 : x = p * q \}$$

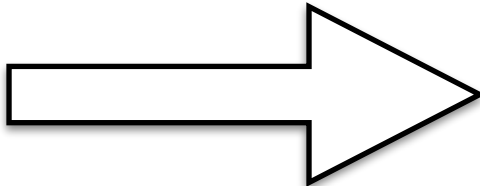
p, q son números con menos tamaño que x y la multiplicación se puede computar en tiempo polinomial. Es decir, (p, q) es un certificado para COMP.

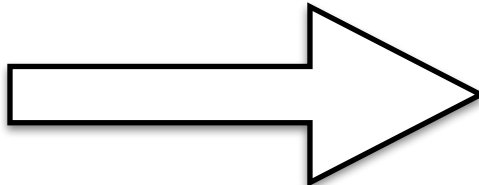
Definición Alternativa de NP

Podemos definir NP como el conjunto de problemas que tienen verificadores polinomiales:

Propiedad: $L \in NP$ si y solo si L tiene un verificador polinomial.

La idea de la demostración es construir una MT no determinista desde un verificador, y viceversa.

Si tiene un verificador  Adivinamos no determinísticamente y verificamos

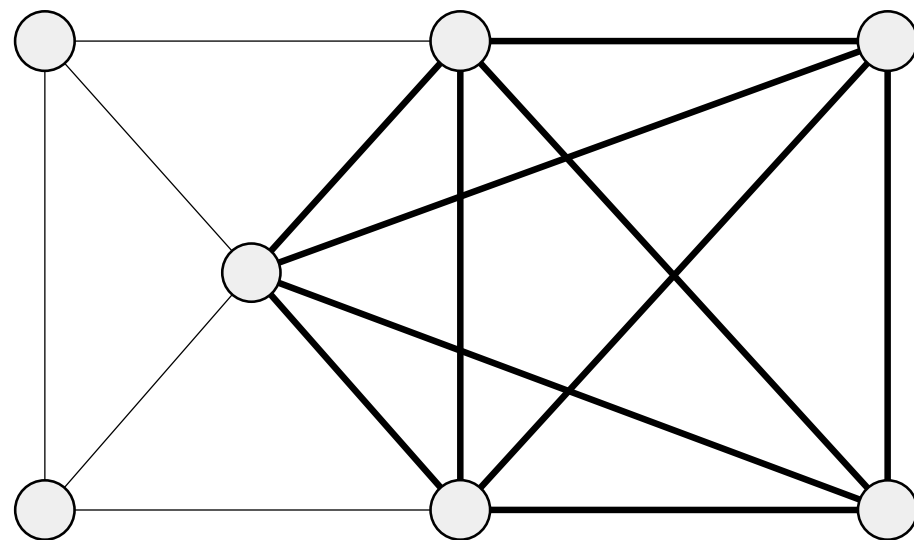
Si tiene un MT no-determinística  En input $\langle w, c \rangle$, c se usa para obtener la rama de ejecución, y se acepta si la MT acepta

Ejemplos de Problemas en NP

Clique

Dado un grafo no dirigido G , un **clique** es un subgrafo de G en el cual cada par de nodos está conectado.

Dado un grafo no dirigido G , un **k-clique** es un clique con k nodos.



Grafo con un 5-clique

$$\text{CLIQUE} = \{ \langle G, k \rangle \mid G \text{ tiene un } k \text{ clique} \}$$

Clique

CLIQUE es en NP , construyamos un verificador para el problema:

Dado $\langle G, c \rangle$:

- ➡ Controlamos que los nodos en c esten en G
- ➡ Controlamos que todos los nodos tengan arcos entre ellos
- ➡ Si las dos condiciones se cumplen se acepta, sino se rechaza.

Notar que c es un subconjunto de nodos, es decir, es de longitud polinomial

Subset-Sum

Dada una colección de números enteros $x_1, \dots, x_k$ y un número t , decir si hay algún subconjunto tal que su suma sea t .

$$\text{SUBSET-SUM} = \{ \langle S, t \rangle \mid \exists S' \subseteq S : \sum S' = t \}$$

Por ejemplo: $\langle \{4, 11, 16, 21, 27\}, 25 \rangle \in \text{SUBSET-SUM}$ ya que:

$$4 + 21 = 25$$

Problemas afuera de NP

Hay muchos problemas que quedan afuera de NP . Por ejemplo:

$$\overline{HAMPATH} = \{w \mid w \notin HAMPATH\}$$

Este es el complemento de HAMPATH, no hay certificados obvios para ver que un grafo no tenga ciclos Hamiltonianos

$$\overline{CLIQUE} = \{w \mid w \notin CLIQUE\}$$

No hay certificados obvios para ver que una grafo no tiene un clique.

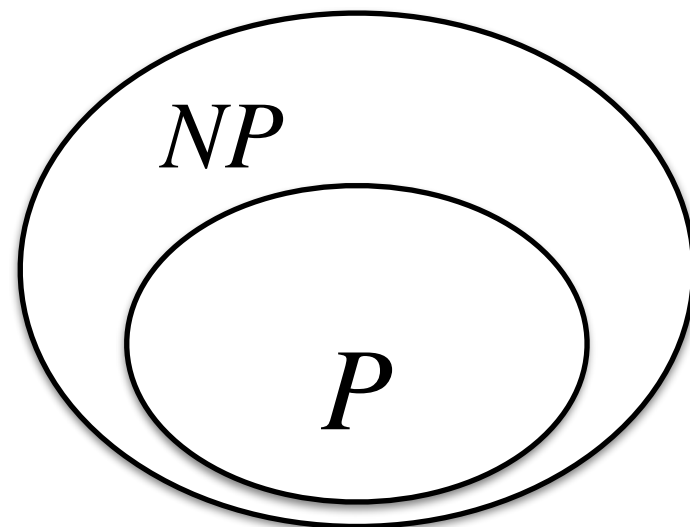
P vs NP

Podemos resumir lo visto de la siguiente forma:

P : lenguajes que pueden ser decididos rápidamente

NP : lenguajes cuya pertenencia puede ser verificada rápidamente

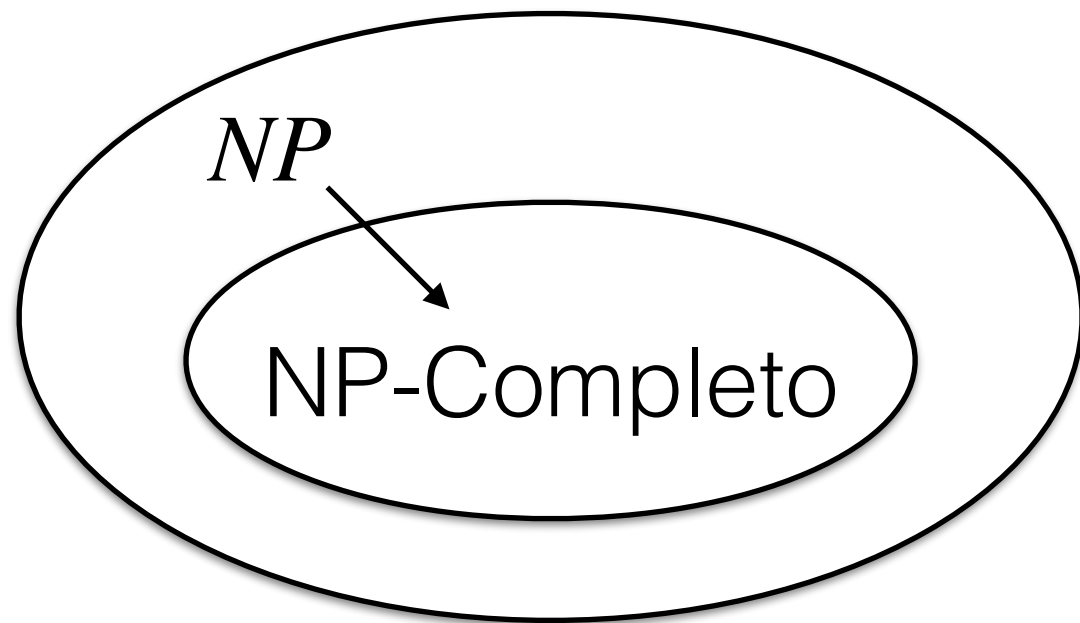
Obviamente, tenemos:



La pregunta si $P=NP$, es una de los grandes problemas no resueltos de computación. A priori la verificabilidad polinomial parece más potente que la decidibilidad polinomial

NP-Compleitud

Los problemas NP-Completo son una subclase de los problemas NP, que permiten resolver cualquiera de los otros problemas NP



- Todos los problemas en NP son reducibles a cualquier problema NP-completo
- Si se encuentra una solución en P para un problema NP-completo, tendríamos que $NP=P$

Reducibilidad Polinomial

Una función $f: \Sigma^* \rightarrow \Sigma^*$ se dice **polinomialmente computable** si existe una MT que corre en tiempo polinomial y compute f .

Un lenguaje A es **polinomialmente reducible** a un lenguaje B , notado $A \leq_p B$ si existe una función polinomialmente computable $f: \Sigma^* \rightarrow \Sigma^*$ tal que para todo w :

$$w \in A \Leftrightarrow f(w) \in B$$

Esto nos permite reducir problemas que no sabemos como solucionar a otros que ya sabemos solucionar.

Propiedades de la Reducibilidad Polinomial

Algunas propiedades de la reducibilidad polinomial:

Propiedad: Si $A \leq_P B$ and $B \in P$, entonces $A \in P$.

Para ver que se cumple consideremos una MT M que computa la función $f : \Sigma^* \rightarrow \Sigma^*$ tal que $w \in A \Leftrightarrow f(w) \in B$. Y una MT M_B que decide B

Diseñemos una MT N para resolver A :

- ➡ En entrada w , se ejecuta M con w se obtiene $f(w)$
- ➡ Se ejecuta M_B que decide B , si acepta, se acepta, sino se rechaza.

M toma $O(n^k)$ pasos y M_B toma $O(n^{k'})$ pasos, luego se toman $O(n^k) + O(n^{k'})$ pasos.

SAT

SAT fue el primer problema NP-completo que se encontró

Fórmulas:

$$\phi, \psi = X \mid \neg\phi \mid \phi \wedge \psi \mid \phi \vee \psi \quad \text{en donde: } X = \{x, y, z, \dots\}$$

Su semántica viene dada por *asignaciones*:

$$v : X \rightarrow \{0,1\} \quad (\text{funciones a valores de verdad})$$

Dada una
asignación
podemos
evaluar una
fórmula:

$(\neg x \wedge y)$			$\vee$	$(x \wedge \neg y)$		
1	0	0	0	0	0	1
0	0	0	1	1	1	1
0	0	1	0	0	0	0
0	0	1	0	1	0	0

Una fórmula es **satisfacible**
si hay una asignación que la
hace verdadera

3-SAT

Llamamos clausula a formulas del estilo:

$$(x_1 \vee \dots \vee x_n)$$

Una formula se dice en CNF (Conjunctive Normal Form) si tiene la forma:

$$c_0 \wedge c_1 \wedge \dots \wedge c_m$$

Todas las fórmulas se pueden escribir en CNF

Es decir, es una conjunción de disyunciones.

Una fórmula se dice en 3NF si está en CNF, y cada cláusula tiene exactamente 3 variables. Por ejemplo:

$$(x_0 \vee x_1 \vee x_3) \wedge (\neg x_0 \vee \neg x_1 \vee x_2)$$

3-SAT a Clique

Veamos cómo podemos reducir 3-SAT a Clique

$$\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$$

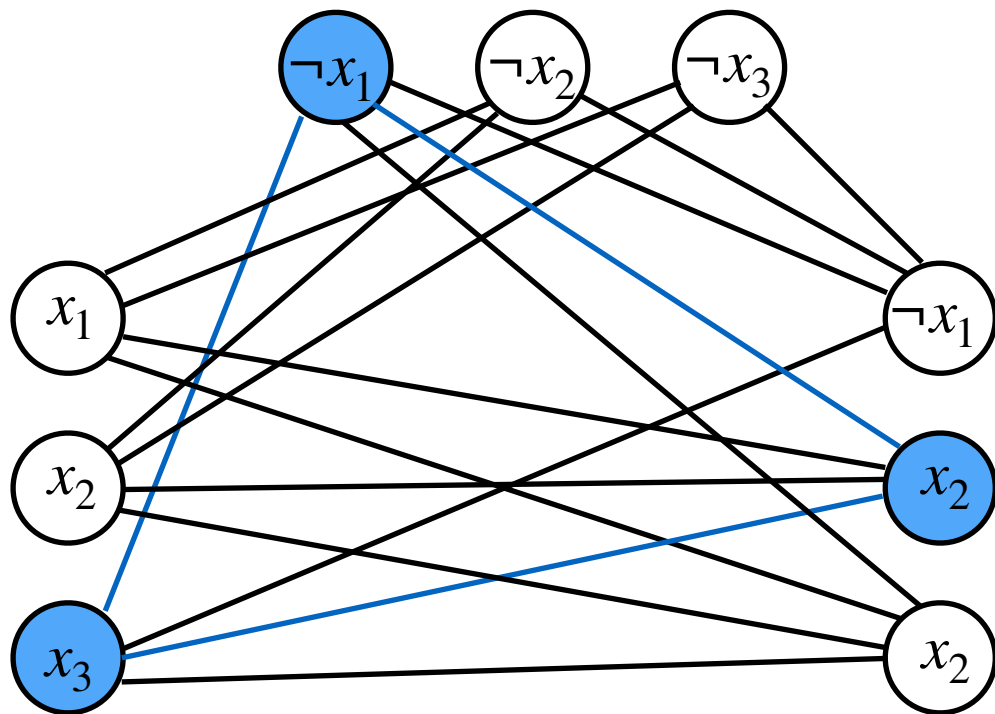
Definimos un grafo G en donde:

- Por cada cláusula tenemos 3 nodos,
- Todos los nodos están conectados entre ellos salvo:
 - Aquellos que son contradictorios ($x, \neg x$),
 - Aquellos que están en la misma cláusula.

Ejemplo

Veamos un ejemplo:

$$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_2)$$



Si hay un clique, podemos asignarle True a los nodos del clique y se hará verdadera la formula

SAT

Podemos definir el problema SAT:

$$\text{SAT} = \{ \langle \phi \rangle \mid \phi \text{ is a satisfiable formula} \}$$

El teorema de Cook-Levin permitió encontrar el primer problema **NP-completo**:

Un problema se dice NP-completo si:

- I. Si está en NP.
- II. Si cualquier problema en NP es reducible polinomialmente a él.

Si algún problema NP-completo está en NP entonces: $\text{NP}=\text{P}$.

Teorema de Cook-Levin

El teorema de Cook-Levin permite demostrar que SAT es NP-completo

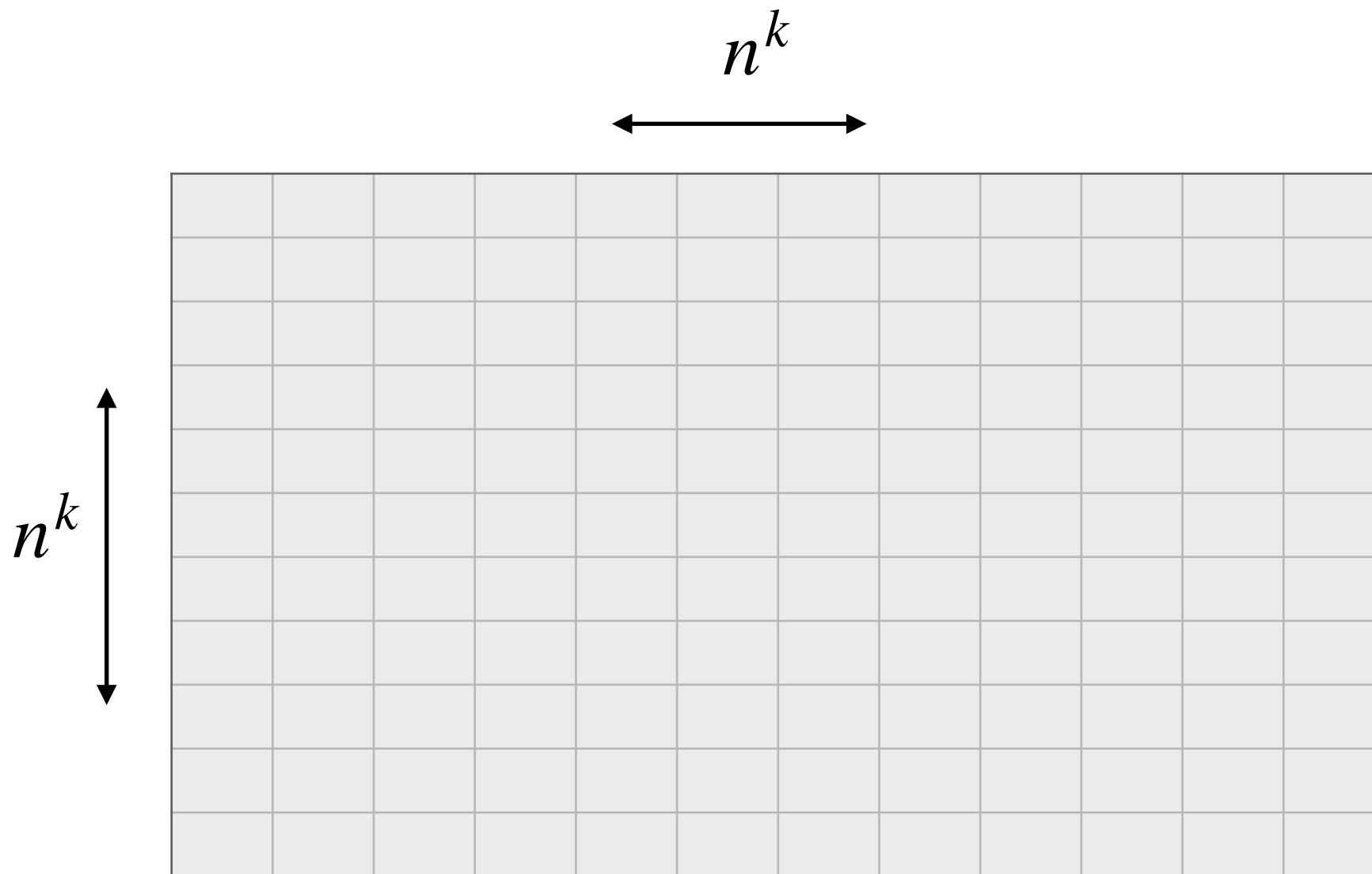
Idea:

1. Ver que SAT está en NP.
2. Demostrar que cualquier problema NP puede ser reducido a SAT:

El punto 2 es el más difícil: dado un problema en NP, que es decidido por una MT N , se construye una fórmula ϕ que es SAT ssi N acepta la entrada. Es importante que ϕ pueda ser construida en tiempo polinomial

Teorema de Cook-Levin

Dada la MT no-determinista N , que hace n^k pasos:
consideramos la siguiente matriz.



Cada fila es un
estado

se pasa de la fila i
a la $i + 1$ si del
estado en la i
existe una
ejecución que se
pasa a la
ejecución $i + 1$

La formula ϕ

Para cada símbolo $s \in C = \Gamma \cup \Sigma \cup Q \cup \{\#\}$ consideramos la variable: $x_{i,j,s}$ que es verdadera cuando $cell[i][j] = s$

$$\phi = \phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$$

Cada una de las partes describe un aspecto de N

$$\phi_{cell} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s, t \in C \\ s \neq t}} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right].$$

en todas las posiciones hay
algún símbolo valido

Puede haber solo un
símbolo

La Formula ϕ

$$\begin{aligned}\phi_{\text{start}} = & x_{1,1,\#} \wedge x_{1,2,q_0} \wedge \\ & x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \dots \wedge x_{1,n+2,w_n} \wedge \\ & x_{1,n+3,\sqcup} \wedge \dots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#} .\end{aligned}$$

En la primera fila hay una configuración inicial

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{\text{accept}}}$$

Alguna de las filas contiene un estado de aceptación

La formula ϕ_{moves} es más complicada, describe los posibles movimientos para eso se usa ventanas de 2×3 celdas

Una ventana es legal si esa configuración puede darse en la matriz.

Ventanas

Las ventanas permiten describir el movimiento de N usando poco espacio (6 celdas)

Por ejemplo, si tenemos:

$$\delta(q_1, a) = \{(q_1, b, R)\}$$
$$\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$$

Las siguientes ventanas serian validas:

(a)

a	q_1	b
q_2	a	c

(b)

a	q_1	b
a	a	q_2

(c)

a	a	q_1
a	a	b

(d)

#	b	a
#	b	a

(e)

a	b	a
a	b	q_2

(f)

b	b	b
c	b	b

Ventanas

Teorema: si todas las ventanas son validas, entonces la matriz describe una rama de la MT.

Si un símbolo está en el centro de una ventana sin ser adyacente a un estado, este se mantiene igual en la próxima fila.

Para aquellas celdas que tienen un estado en el centro, la fila de abajo deberá tener la configuración correspondiente.

Es decir, si todas las ventanas posibles son validas, entonces la matriz es una rama de ejecución de la MT no-determinista.

La Formula ϕ (cont.)

Veamos la formula que identifica los movimientos validos:

$$\phi_{move} = \bigwedge_{0 < i, j < n^k} \text{la ventana centrada en } (i, j) \text{ es legal}$$

Para describir la ventana centrada en (i, j) solo necesitamos hablar de 6 celdas.

Veamos el tamaño de la formula: ϕ_{moves} es tamaño $O(n^{2k})$

ϕ_{start} es tamaño $O(n^k)$

ϕ_{cell} es tamaño $O(n^{2k})$

ϕ_{end} es tamaño $O(n^{2k})$

El tamaño total es polinomial
con respecto a la entrada

3-Sat es NP-Completo

Para ver que 3-sat es NP-completo tenemos que ver:

- Que es NP, tenemos que pensar un algoritmo no-determinista que lo resuelva.
- Que cualquier problema NP se puede reducir polinomialmente a el.
 - ➡ Si reducimos SAT a 3SAT entonces por transitividad de la reducción polinomial obtenemos el resultado.

Reduciendo SAT a 3SAT

En realidad solo tenemos que reducir formulas CNF a 3SAT

Dada una clausula:

$$(x_0 \vee x_1 \vee \dots \vee x_n)$$

La formula usada para ver que SAT es NP se puede fácilmente escribir en CNF

La podemos reescribir con $n - 2$ clausulas:

$$(x_0 \vee x_1 \vee z_1) \wedge (\neg z_1 \vee x_2 \vee z_2) \wedge (\neg z_2 \vee x_3 \vee z_3) \wedge \dots$$

Que tiene $n - 2$ variables nuevas, y está en *3SAT*

Estas dos fórmulas son **equisatisfacibles**.

SAT a 3SAT

Es decir, dada:

$$C_0 \vee C_1 \vee C_{m-1}$$

De longitud N podemos traducirla a 3SAT utilizando N variables nuevas

Por cada variable en C tenemos una clausula con 3 variables. Es decir en total tenemos una fórmula de longitud $O(N)$

Luego chequeamos satisfiabilidad, si la formula $f(C_1 \vee \dots C_{n-1})$ es SAT entonces $C_1 \vee \dots \vee C_{n-1}$ es SAT

Ejercicios:

1. Cuáles de los siguientes son verdaderos:

1. $3^n \in 2^{O(n)}$

2. $2^n \in o(3^n)$

3. $1 \in o\left(\frac{1}{n}\right)$

2. Mostrar que P es cerrado bajo unión, concatenación y complemento

3. Mostrar que NP es cerrado bajo unión, concatenación.

4. Un triángulo es un 3-clique. Dado $\text{Triangle} = \{\langle G \rangle \mid G \text{ contiene un 3-clique}\}$ está en P

5. Mostrar que ALL_{AFD} está en P

6. Dos grafos son isomorficos si existe un isomorfismo entre ellos. Demostrar que ISO el problema de decir si dos grafos son iso o no está en NP

Problemas NP-Completo

El problema del cubrimiento de vertices

Dado un grafo $G = \langle V, E \rangle$ decir si existen $k \leq |V|$ vertices tal que cubren el grafo: es decir, **cada arco de G toca algunos de esos nodos.**

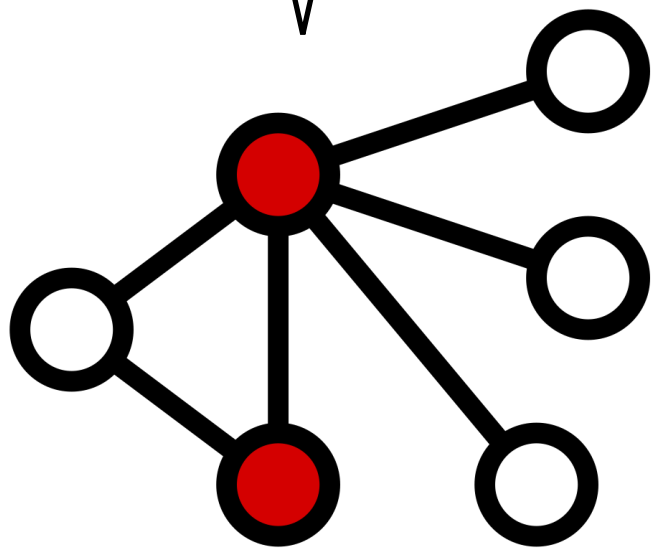
$\text{VERTEX-COVER} = \{ \langle G, k \rangle \mid G \text{ es un grafo no-dirigido que tiene un cubrimiento de } k \text{ nodos} \}$

Veamos que este problema es NP-Completo:

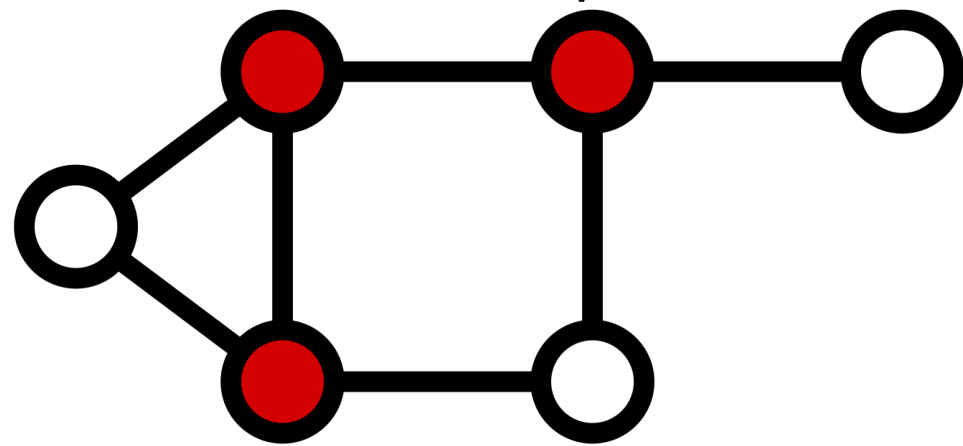
- Tenemos que ver que es NP,
- Tenemos que ver que si un problema A es NP, entonces se reduce polinomialmente a él.

Cubrimiento de Vertices

Tiene un cubrimiento con
dos vertices



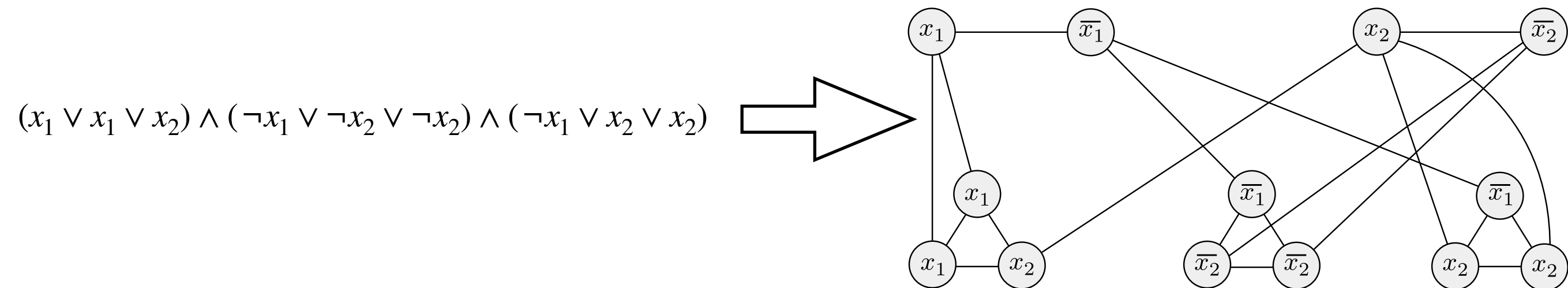
Tiene un cubrimiento con
tres vertices



Idea de la Reducción

Dada la formula: $C_1 \vee \dots \vee C_n$ en 3-CNF

- Creamos un clique con cada cláusula, en donde tenemos un nodo por variable,
- Por cada variable creamos un subgrafo con dos nodos x y $\neg x$ unidos con un arco y unidos con los nodos con el mismo label.



Corrección de la Reducción

- El grafo tiene $2m + 3\ell$ nodos, en donde ϕ tiene m variables y ℓ cláusulas.
- Buscamos un cubrimiento con $m + 2\ell$ nodos (es decir, de las variables tomamos uno, y de las cláusulas tomamos dos).
- Si ν es una asignación entonces podemos encontrar un cubrimiento. Dada una asignación, elegimos una variable verdadera de cada cláusula, y los nodos identificados con las dos restantes la ponemos en cubrimiento
- Si tenemos un cubrimiento podemos obtener una asignación que satisface la fórmula. Dado un cubrimiento, este debe tener dos nodos de cada cláusula y uno de cada variable, sino no sería un cubrimiento válido. Se seleccionan las variables correspondientes a las conexiones de variables.

Camino Hamiltoniano

Veamos que es NP-completo:

- Ya vimos que está en NP.
- Para ver que es NP-completo reducimos polinomialmente un problema NP-completo a HAMPATH

Vamos a reducir 3-SAT a HAMPATH. La idea es dada una fórmula en 3CNF, construir un grafo que tenga un camino Hamiltoniano si la fórmula es SAT.

Ejercicios

1. Dos grafos G y F son isomorfos si los nodos de G pueden ser reordenados tal que G y F son identicos. Mostrar que este problema está en NP.
2. Mostrar que $MODEXP = \{ \langle a, b, c, p \rangle \mid a, b, c, p \text{ son enteros positivos y } a^b \equiv_p c \}$ está en P.
3. Mostrar que NP es cerrado bajo $*$.
4. Mostrar que si $NP=P$ entonces cualquier lenguaje $A \in P$ (excepto $A = \emptyset$ y $A = \Sigma^*$) es NP-completo.
5. Mostrar que $LPATH = \{ \langle G, a, b, k \rangle \mid G \text{ contiene un camino simple de longitud mas grande que } k \text{ de } a \text{ a } b \}$ es NP-Completo
6. Mostrar que $3COLOR = \{ \langle G \rangle \mid G \text{ is coloreable con tres colores} \}$ es NP-completo.
7. Qué sucede con $KCOLOR$?

Complejidad Espacial

Cuando analizamos algoritmos hay dos dimensiones importantes:

Tiempo: cantidad de instrucciones básicas que se realizan cuando se ejecuta un algoritmo.

Espacio: cantidad de memoria que necesita un algoritmo para resolver un problema.

Siempre utilizaremos el modelo de máquinas de Turing. En el caso de la memoria mediremos cuantas celdas de la cinta necesita la MT durante su ejecución.

Complejidad Espacial

Más formalmente:

Sea M una MT que siempre termina. La **complejidad espacial** de M es dada por una función $f : \mathbb{N} \rightarrow \mathbb{N}$, donde $f(n)$ es la **cantidad máxima de celdas** que M mira para cualquier entrada de longitud n ,

Si M es no determinista $f(n)$ es la cantidad máxima de **celdas inspeccionadas en cualquier rama**.

Clases de Complejidad de Espacio

Para definir las clases de complejidad en espacio utilizamos la notación O grande:

$$SPACE(f(n)) = \{L \mid L \text{ es decidido por una MT en espacio } O(f(n))\}$$

$$NSPACE(f(n)) = \{L \mid L \text{ es decidido por una MT no det. en espacio } O(f(n))\}$$

De la misma forma podemos definir:

$$PSPACE = \cup_{k \geq 1} SPACE(O(n^k))$$

$$NPSPACE = \cup_{k \geq 1} NSPACE(O(n^k))$$

Ejemplo

Los algoritmos pueden reusar espacio pero no pueden reusar el tiempo.

Veamos que SAT está en *PSPACE*

Sea M la MT tal que dado el input ϕ :

Solo necesita una cantidad
linear de espacio l

1. Para cada asignación de valores de verdad de ϕ
2. Evaluar ϕ en la asignación:
 - ➡ Si se evalua a true, aceptar.
 - ➡ Sino limpiar la porción de memoria usada y continuar
3. Rechazar.

NPSPACE

NPSPACE es el analogo no determinista a PSPACE:

Hagamos una MT que decida

Solo necesita una cantidad
linear de espacio l

$$\overline{ALL_{NFA}} = \{ \langle A \rangle \mid A \text{ es un NFA y } L(A) \neq \Sigma^* \}$$

N = en input $\langle M \rangle$ con M un NFA:

1. Poner una marca en el estado inicial

2. Repetir 2^Q veces:

3. no determinísticamente elegir un símbolo, y
cambiar la marcas para simular M

2. Si no se marcaron estados de aceptación, aceptar, sino
rechazar.

Si hay una cadena que
no se acepta, hay una
cadena de longitud 2^Q
no aceptada

El Teorema de Savitch

El teorema de Savitch dice que:

$$\text{NSPACE}(f(n)) \subseteq \text{SPACE}(f(n)^2)$$

Para ver esto tenemos que, dada un MT no determinista que corre utiliza a lo sumo $f(n)$ celdas, simularla con una MT determinista que use a lo sum $f(n)^2$ celdas,

Una propiedad interesante es la siguiente:

No podemos simular el árbol porque cada rama podría tener una long. exponencial

Una MT que usa a lo sumo $O(f(n))$ espacio, puede realizar a lo sumo $f(n)2^{O(f(n))}$ pasos

Teorema de Savitch

La prueba consiste en definir un programa:

`canyield(c1, c2, t)`

que dice si en t pasos podemos ir de la configuración $c1$ a la configuración $c2$, que utilice poco espacio.

- La idea es que use $f(n)^2$ espacio,
- Es un algoritmo recursivo, tenemos que llevar la pila de recursión
- Cada configuración ocupa a lo sumo $O(f(n))$ espacio

El algoritmo canyield

canyield($c1, c2, t$) :

- 1 Si $t=1$, se verifica si $c1=c2$ o si puedo llegar en un paso a $c2$, y se acepta o rechaza de acuerdo a esto.
- 2 Si $t > 1$ para cada configuración cm
 - 3 ejecutamos canyield($c1, cm, t/2$)
 - 4 ejecutamos canyield($cm, c2, t/2$)
 - 5 si los pasos 3 y 4 aceptan, aceptamos
 - 6 sino se acepto para ninguna se rechaza

Análisis del algoritmo

Corremos $\text{canyield}(c0, \text{accept}, 2^{df(n)})$ para alguna constante d

- La MT original puede realizar a lo sumo $2^{O(f(n))}$ pasos, es decir si acepta el algoritmo aceptará
- La pila de recursión solo puede tener tamaño logarítmico, (el t se divide por dos en cada recursión)
- En cada llamada utilizamos $f(n)$ celdas para la configuración más $\log 2^{df(n)}$ espacio para la pila de recursión, es decir, nos da $f(n)^2$ celdas

Consecuencias del Teorema de Savitch

Una primera consecuencia es la siguiente:

$$NPSPACE = PSPACE$$

Hasta ahora obtenemos:

$$P \subseteq NP \subseteq PSPACE = NPSPACE \subseteq EXPTIME$$



Esto es debido a la propiedad que vimos antes

PSPACE Completeness

Al igual que definimos problemas NP-completos podemos definir problemas PSPACE-completos:

Un problema B se dice PSPACE-Completo si:

1- $B \in PSPACE$

2- para todo problema $A \in PSPACE$ tenemos que
 $A \leq_P B$

Nota que también utilizamos reducibilidad polinomial, si tenemos un procedimiento que corre en tiempo polinomial también corre en espacio polinomial.

Ejercicios

- Mostrar que la clase PSPACE es la misma si se usan MT con una cinta o dos cintas. Qué sucederá con k cintas?
- Mostrar que PSPACE es cerrado bajo la unión, la complementación y la estrella.
- Demostrar que cualquier lenguaje PSPACE-Completo es NP-hard.
- Demostrar que si cualquier lenguaje NP-hard es también PSPACE-hard, entonces $NP=PSPACE$
- Mostrar que 3-color es NP-completo.
- Mostrar que Sudoku es NP-completo.